

DATABASE MANAGEMENT SYSTEMS
UNIT II – TOPIC 1
RELATIONAL MODEL

What is Relational Model?

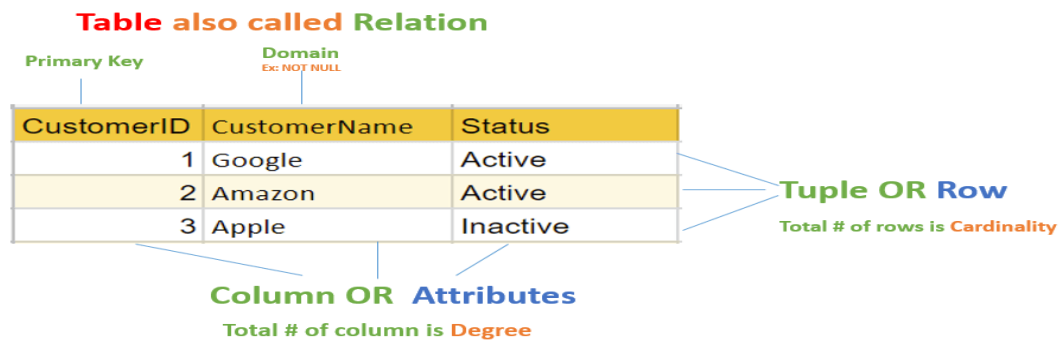
Relational model can represent as a table with columns and rows. Each row is known as a tuple. Each table of the column has a name or attribute.

Relational Model (RM) represents the database as a collection of relations. A relation is nothing but a table of values. Every row in the table represents a collection of related data values. These rows in the table denote a real-world entity or relationship.

The table name and column names are helpful to interpret the meaning of values in each row. The data are represented as a set of relations. In the relational model, data are stored as tables. However, the physical storage of the data is independent of the way the data are logically organized.

Relational Model – Keywords / Terminologies

1. **Attribute:** Each column in a Table. Attributes are the properties which define a relation. e.g., Student_Rollno, NAME, etc.
2. **Tables** – In the Relational model the, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.
3. **Tuple** – It is nothing but a single row of a table, which contains a single record.
4. **Relation Schema:** A relation schema represents the name of the relation with its attributes.
5. **Degree (Arity):** The total number of attributes which in the relation is called the degree of the relation.
6. **Cardinality:** Total number of rows present in the Table.
7. **Column:** The column represents the set of values for a specific attribute.
8. **Relation instance** – Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.
9. **Relation key** - Every row has one, two or multiple attributes, which is called relation key.
10. **Attribute domain** – Every attribute has some pre-defined value and scope which is known as attribute domain
11. **Relational database** – A collection of relations with distinct relation names
12. **Relational database schema** - The collection of schemas for the relations in the database.



Relational Model Concepts:

1. **Data is Organized into Tables:** Data is organized into tables, also known as relations. Each table consists of rows and columns. Rows represent individual records, while columns represent attributes or fields.
2. **Tables Have Keys:** Each table has one or more columns that uniquely identify each row. These are called primary keys. Additionally, there can be foreign keys, which establish relationships between tables.
3. **Relationships Between Tables:** Tables can be related to each other through common attributes. These relationships can be one-to-one, one-to-many, or many-to-many.
4. **Data Integrity:** The relational model enforces data integrity through constraints such as primary key constraints, foreign key constraints, and other integrity rules. These constraints ensure that data remains consistent and accurate.
5. **Structured Query Language (SQL):** The relational model is typically manipulated using a special-purpose language called SQL (Structured Query Language). SQL provides a standardized way to perform operations such as querying, updating, and deleting data in relational databases.
6. **Normalization:** The process of organizing data to minimize redundancy and dependency is called normalization. It involves breaking down large tables into smaller ones and defining relationships between them to reduce redundancy and improve data integrity.
7. **Transactions:** Relational databases support transactions, which are units of work that must be performed atomically (all or nothing), consistently (in a valid state), isolated (separate from other transactions), and durably (persistently stored).

DATABASE MANAGEMENT SYSTEMS

UNIT II – TOPIC 1

RELATIONAL MODEL

Structured query language (SQL) is a programming language for storing and processing information in a relational database.

We can use SQL statements to store, update, remove, search, and retrieve information from the database and also to maintain and optimize database performance.

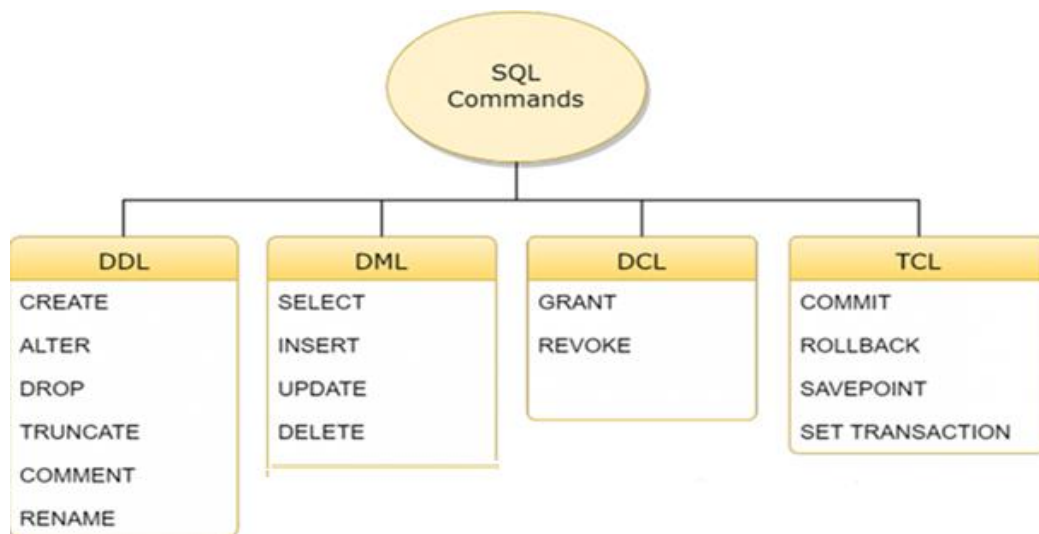
SQL was invented in the 1970s based on the relational data model. It was initially known as the Structured English Query Language (SEQUEL).

This query language became the standard of ANSI in the year of 1986 and ISO in the year of 1987.

Types of SQL Commands:

Here are five types of widely used SQL queries.

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
- Data Query Language
- Data Control Language (DCL)
- Transaction Control Language (TCL)



What is DDL?

Data definition language (DDL) refers to SQL commands that design the database structure. i.e. it deals with **database schemas and descriptions**, of how the data should reside in the database. Database engineers use DDL to create and modify database objects based on the business requirements. For example, the database engineer uses the CREATE command to create database objects such as tables, views, and indexes.

CREATE:

To create a database and its objects like (table, index, views, store procedure, function, and triggers)

Syntax: CREATE Objtype Objname;

CREATE DATABASE

Syntax : create database database_name;

Ex: create database testdb;

Use the SHOW statement to find out what databases currently exist on the server:

Syntax: SHOW DATABASES;

Ex: show databases;

We have to select the database on which we want to run the database queries.

syntax : USE DATABASE_NAME;

Ex: use testdb;

Creating a Table:

The CREATE TABLE statement is used to create a new table in a database.

Syntax:
CREATE TABLE table_name (
column1 datatype(size),
column2 datatype(size),
column3 datatype(size),
....
);

Example: create table emp (eid int(5), ename varchar(15));

The column parameters specify the names of the columns of the table.

The datatype parameter specifies the type of data the column can hold
(e.g. varchar, integer, date, etc.)

Create Table using Another Table :

A copy of an existing table can also be created using CREATE TABLE.

- The new table gets the same column definitions. All columns or specific columns can be selected.
- If you create a new table using an existing table, the new table will be filled with the existing values from the old table.

Syntax:

CREATE TABLE new_table_name AS
SELECT column1, column2,...
FROM existing_table_name
WHERE;

Example: The following SQL creates a new table called "DupCust" (which is a copy of the "Customers" table):

```
CREATE TABLE DupCust AS
SELECT customername, contactno
FROM customers;
```

Data Types in MySQL:

In MySQL there are three main data types: **string, numeric, and date and time**

String Data Type:

The attribute domain includes characters – alphabets, numerals and special characters.

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The <i>size</i> parameter specifies the column length in bytes. Default is 1

Numeric Data Type:

Data type	Description
BIT(size)	A bit-value type. The number of bits per value is specified in size. The size parameter can hold a value from 1 to 64. The default value for size is 1.
BOOL / BOOLEAN	Zero is considered as false, nonzero values are considered as true.
INT(size) / INTEGER(size)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The size parameter specifies the maximum display width (which is 255)
FLOAT(size, d)	A floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
DOUBLE(size, d)	A normal-size floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter
DECIMAL(size, d) / DEC(size, d)	An exact fixed-point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter. The maximum number for size is 65. The maximum number for d is 30. The default value for size is 10. The default value for d is 0.

Date and Time Datatype:

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(fsp)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(fsp)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC.
TIME(fsp)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.

DROP

Drops commands remove tables and databases from RDBMS.

To drop a Table

Syntax: DROP TABLE tablename;

Ex: DROP TABLE emp ;

To drop a database

syntax: DROP DATABASE databasename;

Ex: Drop database kmitcsma;

TRUNCATE:

This command is used to delete all the rows from the table and free the space containing the table.

Syntax:

TRUNCATE TABLE table_name;

Example:

TRUNCATE table students

ALTER:

1. The ALTER TABLE statement is used to modify the structure of table, i.e. **add, delete, or modify columns** in an existing table.
2. The ALTER TABLE statement is also used to add and drop various constraints on an existing table.

ALTER- ADD

ALTER- MODIFY

ALTER-DROP

To add a new column to the Table :

Syntax:

```
ALTER TABLE table_name ADD (columnname-1 datatype(size),  
columnname-2 datatype(size)...,columnname-n datatype(size));
```

Ex: Alter Table emp add (dob date, mobile int(10));

To Drop a column from an existing table:

Syntax:

```
ALTER TABLE tablename DROP column columnname;
```

Ex: alter table emp drop column mobile;

To modify a column in the Table :

Syntax:

```
ALTER TABLE table_name MODIFY columnname-1 datatype(size);
```

Ex: Alter Table emp modify ename varchar(25);

To add a constraint:

Syntax:

```
ALTER TABLE tablename ADD constrainttype (columnname);
```

Ex: alter table emp add primary key(eid);

RENAME:

Renaming the existing table:

```
RENAME TABLE table_name to new table_name;
```

Ex:

```
mysql> rename table persons to emp;  
Query OK, 0 rows affected (3.17 sec)
```

Renaming the column name in an existing table:

```
ALTER TABLE table_name  
RENAME COLUMN old_name TO new_name;
```

Ex: ALTER TABLE student1
RENAME COLUMN jdateTO joingdate;

DESCRIBE :

This command is used to describe the structure of an existing table

Knowing the structure of a table:

DESCRIBE table_name;

OR

DESC table_name

Ex: mysql> create table emp (eid int(5), ename varchar(15));
Query OK, 0 rows affected, 1 warning (0.06 sec)
mysql> describe emp;

```
mysql> desc emp;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| eid   | int           | YES  |     | NULL    |       |
| ename | varchar(15)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

COMMENT:

Comments are used to explain sections of SQL statements.

Comments can be written in the following three formats:

- 1.Single-line comments
- 2.Multi-line comments
- 3.In-line comments

1.Single Line Comments

Comments starting and ending in a single line are considered single-line comments. A line starting with ' - ' is a comment and will not be executed.

Ex: create database testdb; --THIS IS SINGLE LINE COMMENT:

2. Multi-Line Comments

Comments starting in one line and ending in different lines are considered as multi-line comments.

A line starting with '/*' is considered as starting point of the comment and is terminated when '*/' is encountered.

**Ex: /*This is for multiline comments
specifying the database is created:*/**

Create database kmittest;

3. In-Line Comments

In-line comments are an extension of multi-line comments, comments can be stated in between the statements and are enclosed in between ‘/*’ and ‘*/’. Bypassed by /

EX:

SELECT ename

/* This column contains the name of the customer / order_date /

This column contains the date the order was placed */

FROM emp

DATABASE MANAGEMENT SYSTEMS

UNIT II – TOPIC 3

INTEGRITY CONSTRAINTS

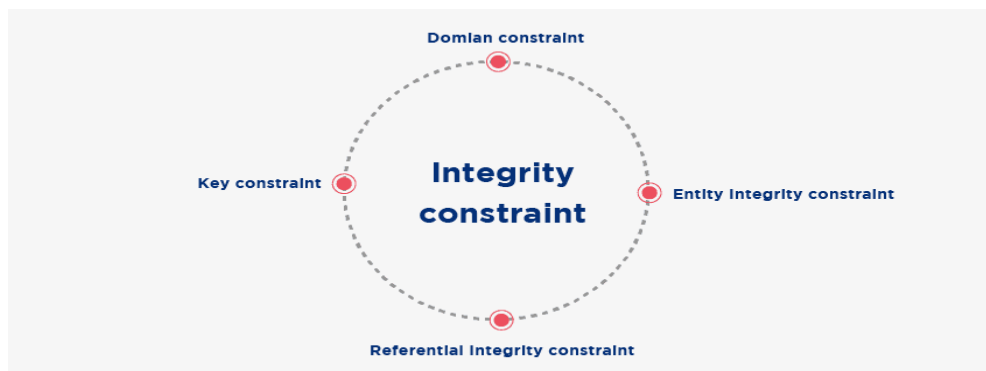
Integrity constraints are the set of predefined rules in a database to maintain data accuracy, consistency and reliability. In short, it is used to maintain the quality of information. They can be used to enforce business rules or to ensure that data is entered correctly. Integrity constraints can also be used to enforce relationships between tables. They ensure that the data in the database adheres to a set of rules, which can help prevent errors and inconsistencies.

Integrity Constraints act as guidelines ensuring that data in the database remain accurate and consistent. They guard against accidental damage to the database.

Integrity constraints in SQL can be either enforced by the database system or by application code. Enforcing them at the database level can help ensure that the rules are always followed, even if the application code is changed. However, enforcing them at the application level can give the developer more flexibility in how the rules are enforced.

Types of Integrity Constraint

There are four main types of integrity constraints: domain, entity, referential, and key.



1. Domain constraints

Domain constraints can be defined as the definition of a valid set of values for an attribute. The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

Eid	Ename	Job	Sal	Location	Deptno
100	Rama	Scientist	102000	Hyd	12
101	Srikar	Manager	80300	Hyd	20
102	Krishana	Analyst	2Lakhs	Sec	24

Here for Eid 102, Salary attribute violates the domain constraint as it is allowed to have only numeric values.

2. Entity integrity constraints

An entity integrity constraint is a restriction on null values. Null values are values that are unknown or not applicable, and they can be problematic because they can lead to inaccurate results. This constraint serves unique identification of each tuple in the relation.

The primary key attribute is used to identify individual rows in relation and entity integrity constraint states that it cannot be NULL. However a table can contain a NULL value in other fields / attributes.

In the employee table below Eid is primary key and can't contain NULL value.

Eid	Ename	Job	Sal	Location	Deptno
100	Rama	Scientist	102000	Hyd	12
101	Srikar	Manager	80300	Hyd	20
	Krishana	Analyst	95600	Pune	24

3. Referential Integrity Constraints (Foreign Key)

A referential integrity constraint is used to enforce relationships between tables. The foreign key constraint is a column or list of columns that points to the primary key column of another table.

In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

The rules are:

- can't delete a record from a primary table if matching records exist in a related table.
- can't change a primary key value in the primary table if that record has related records.
- can't enter a value in the foreign key field of the related table that doesn't exist in the primary key of the primary table.
- However, you can enter a Null value in the foreign key, specifying that the records are unrelated.

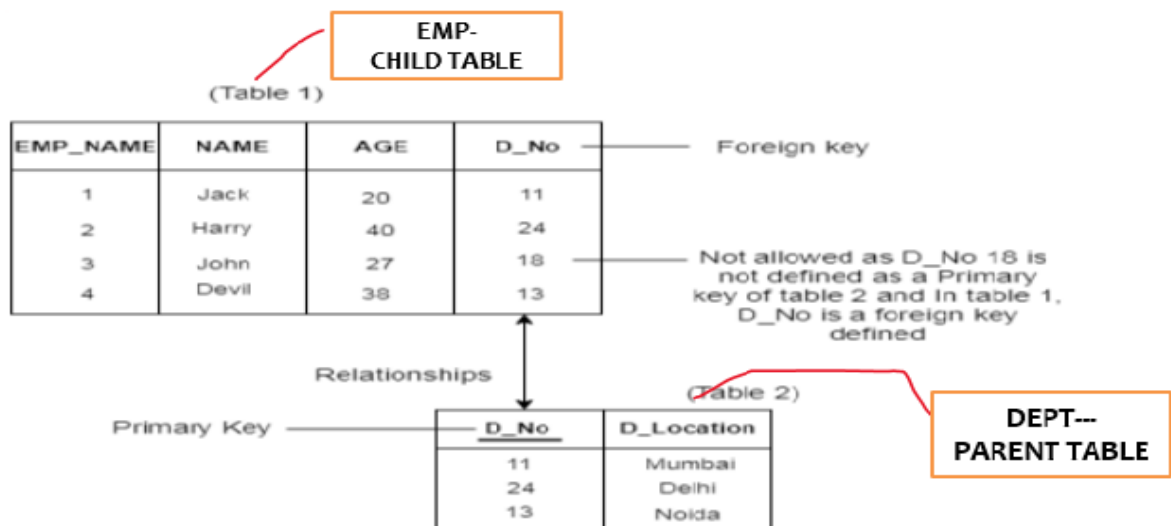
Participation Constraint:

A participation constraint specifies whether all or only some entities of an entity set must participate in a relationship in an ER model.

Total Participation

Partial Participation

Example:



4. Key Constraints

Key constraints in DBMS are a restriction on duplicate values. A key is composed of one or more columns whose values uniquely identify each row in the table. These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.

Keys are the entity set that is used to identify an entity within its entity set uniquely. A entity set can have multiple keys or candidate keys (minimal superkey), out of which we choose one of the keys as the primary key. Primary key is always unique and cannot have null values in the relational table.

Example:

In the below example Eid is primary key but Eid have the same value 100. so, it is violating the key constraint

Eid	Ename	Job	Sal	Location	Deptno
100	Rama	Scientist	102000	Hyd	12
101	Srikar	Manager	80300	Hyd	20
100	Krishana	Analyst	95600	Pune	24

Other Key Constraints:

1. NOT NULL Constraints
2. NULL Constraints
3. PRIMARY KEY Constraints
4. FOREIGN KEY Constraints
5. UNIQUE Constraints
6. CHECK Constraints

NOT NULL Constraint:

The NOT NULL constraint in SQL is used to ensure that a column in a table doesn't contain NULL (empty) values, and prevent any attempts to insert or update rows with NULL values.

The NOT NULL constraint enforces a column to NOT accept NULL values, which means that you cannot insert a new record, or update a record without adding a value to this field.

NULL represents a record where data may be missing data or data for that record may be optional. By default a column is allowed to have NULL values, but the NOT NULL constraint enforces a column to NOT accept NULL values

A not-null constraint cannot be applied at table level

NOT NULL on CREATE TABLE

```
CREATE TABLE Persons (  
    PID int (3) NOT NULL,  
    PName varchar (10) NOT NULL,  
    PADD varchar (20) NOT NULL,  
    PAge int (2)  
);
```

NULL Constraint:

- By default, a column can hold NULL values

The term **NULL** in SQL is used to specify that a data value does not exist in the database. It is not the same as an empty string or a value of zero, and it signifies the absence of a value or the unknown value of a data field.

Some common reasons why a value may be NULL –

- The value may not be provided during the data entry.
- The value is not yet known.

It is important to understand that you cannot use comparison operators such as "=", "<", or ">" with NULL values. This is because the NULL values are unknown and could represent any value. Instead, you must use "IS NULL" or "IS NOT NULL" operators to check if a value is NULL

Syntax

The basic syntax of **NULL** while creating a table.

```
CREATE TABLE CUSTOMERS (  
    ID INT(3) NOT NULL,  
    NAME VARCHAR (20) NOT NULL,  
    AGE INT(2) NOT NULL,  
    ADDRESS VARCHAR (25),  
    SALARY FLOAT(6, 2),  
);
```

In the above example SALARY and ADDRESS are NULL constraints.

PRIMARY KEY Constraint :

The **PRIMARY KEY** constraint uniquely identifies each record in a table.

- Primary keys must contain UNIQUE values, and cannot contain NULL values.
- A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields).

```
CREATE TABLE Persons(  
    PID int(3) NOT NULL,  
    PName varchar(10) NOT NULL,  
    PADD varchar(20) NOT NULL,  
    PAge int (2),  
    PRIMARY KEY (PID)  
);
```

FOREIGN KEY Constraint :

The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables.

A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY in another table.

FOREIGN KEY constraints enforce referential integrity.

The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table.

The rules are:

- can't delete a record from a primary table if matching records exist in a related table.
- can't change a primary key value in the primary table if that record has related records.
- can't enter a value in the foreign key field of the related table that doesn't exist in the primary key of the primary table.
- However, you can enter a Null value in the foreign key, specifying that the records are unrelated.

```
CREATE TABLE dept (  
    deptno int(3),  
    dname varchar(20),  
    location varchar(20),  
    Primary key(deptno) );    -- PARENT TABLE OR Primary table
```

```
CREATE TABLE Emp (  
    eid int(3),  
    ename varchar(20),  
    salary float(6,2),  
    no int(3),  
    PRIMARY KEY (Eid),  
    FOREIGN KEY (dno) REFERENCES dept(deptno) );    -- CHILD TABLE
```

UNIQUE Constraint:

The UNIQUE constraint in SQL is used to ensure that no duplicate values will be inserted into a specific column or combination of columns that are participating in the UNIQUE constraint and not part of the PRIMARY KEY.

In other words, the index that is automatically created when you define a UNIQUE constraint will guarantee that no two rows in that table can have the same value for the columns participating in that index, with the ability to insert only NULL value to these columns.

The UNIQUE constraint ensures that all values in a column are different.

Both the UNIQUE and PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns

A **PRIMARY KEY** constraint automatically has a **UNIQUE** constraint.

Difference between UNIQUE constraints and PRIMARY KEY constraint is unique allows NULL values whereas primary key not allowed the NULL values.

```
CREATE TABLE Persons(  
    PID int(3) UNIQUE,  
    PName varchar(10) NOT NULL,  
    PADD varchar(20),  
    PAge int (2),  
    Padhar varchar(12)  
    PRIMARY KEY (Padhar) );
```

CHECK Constraint:

- The CHECK constraint is used to limit the value range that can be placed in a column.
- If you define a CHECK constraint on a column it will allow only certain values for this column.
- If you define a CHECK constraint on a table it can limit the values in certain columns based on values in other columns in the row.

The following SQL creates a CHECK constraint on the "Age" column when the "Persons" table is created. The CHECK constraint ensures that the age of a person must be 18, or older:

```
CREATE TABLE Persons (  
    PID int(3),  
    PName varchar(10) NOT NULL,  
    PAdd varchar(20) NOT NULL,  
    PAge int (2),  
    PRIMARY KEY(PID),  
    CHECK (PAge>=18) );
```

If we enter the Page below 18 then shows that CHECK Constraint is violated.

```
CREATE TABLE Emp(  
    EID int(3),  
    EName varchar(20) NOT NULL,  
    Doj date,  
    Mobile bigint CHECK(Length(Mobile)=10),  
    PRIMARY KEY(PID) );
```

Check constraint here ensures that mobile number is exactly 10 digits.

DATABASE MANAGEMENT SYSTEMS
UNIT II – TOPIC 4
LOGICAL DATABASE DESIGN

ER Diagram to Relational Model Conversion

The ER model is convenient for representing an initial, high-level database design. First step of any relational database design is to make ER Diagram for it and then convert it into relational Model.

Given an ER diagram describing a database, there is a standard approach to generating a relational database schema that closely approximates the ER design.

What is relational model?

Relational Model represents how data is stored in database in the form of table



Lets learn step by step how to convert ER diagram into relational model.

Logical database design is the process of mapping or translating the conceptual model (ER Diagrams) into a relational model (table). i.e. deciding how to arrange the attributes of the entities in a given business environment into database structures, such as the tables of a relational database.

Let us see how to translate an ER diagram into a collection of tables with associated constraints, i.e., a relational database schema.

Three basic rules to convert ER into tables or relations:

Rule 1: *Entity*: Entity Names will automatically be table names

Rule 2: *Mapping of attributes*: attributes will be columns of the respective tables.

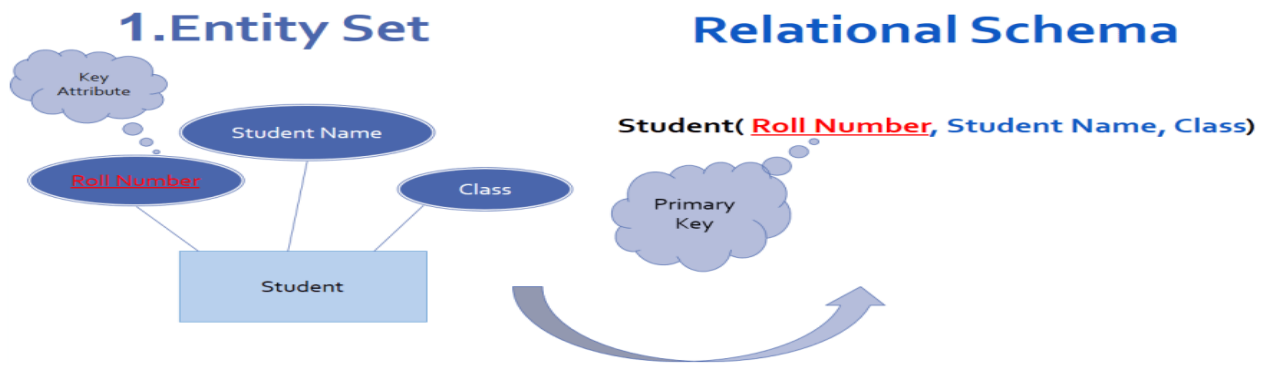
Rule 3: *Relationships*: relationship will be mapped by using a foreign key attribute. Foreign key is a primary or candidate key of one relation used to create association between tables.

1. Entity Set to Table:

Consider we have entity STUDENT in ER diagram with attributes Roll Number, Student Name and Class.

To convert this entity set into relational schema

1. Entity is mapped as relation in Relational schema
2. Attributes of Entity set are mapped as columns for that Relation.
3. Key attribute of Entity becomes Primary key for that Relation.



2. Entity Set with Multivalued Attributes:

An entity may have several attributes which can take more than one value (eg: mobile number, hobby etc.). As it is not possible to represent multiple values in a single column in a relation, separate relation is created for multivalued attribute along with the key attribute of the relation.

1. Key attribute and multivalued attribute of entity set becomes composite key of new relation.
2. Separate relation employee is created with remaining attributes.

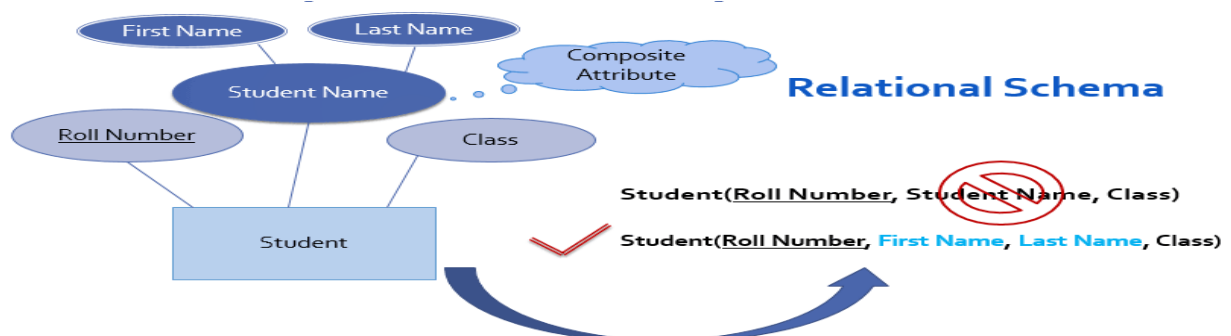
This ensures instead of repeating all attributes of entity (for every value of the multivalued attribute) now only one attribute is need to repeat and that is the key attribute.



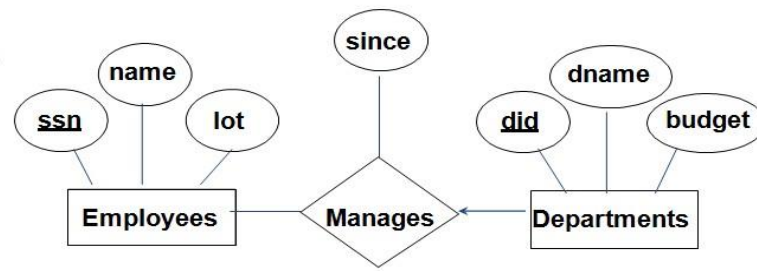
3. Entity set with Composite attribute:

Consider entity set student with attributes Roll Number, Student Name and Class. Here student name is composite attribute as it has further divided into First name, last name.

In this case to convert entity into relational schema, composite attribute student name should not be include in relation but all parts of composite attribute are mapped as simple attributes for relation.



Relationship Sets (with Key Constraints) to Tables



Because each department has at most one manager, no two tuples can have the same *did* value but differ on the *ssn* value.

A consequence of this observation is that *did* is itself a key for Manages

The Manages relation can be defined using the following SQL statement:

```
CREATE TABLE Manages(
    ssn CHAR(11), did INTEGER, since DATE,
    PRIMARY KEY (did),
    FOREIGN KEY (ssn) REFERENCES Employees (ssn),
    FOREIGN KEY (did) REFERENCES Departments(did));
```

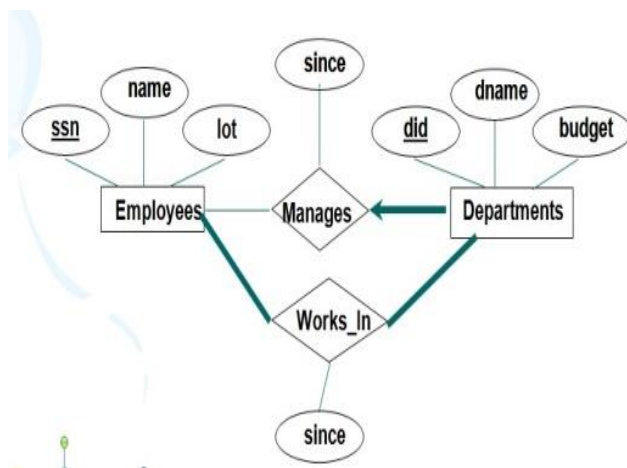
- **A second approach:** Since each department has a unique manager, we could instead combine Manages and Departments.
- This approach eliminates the need for a separate Manages relation, and queries asking for a department's manager can be answered without combining information from two relations.
- The only drawback to this approach is that space could be wasted if several departments have no managers.
- In this case the added fields would have to be filled with *null* values.

The Dept_Mgr relation can be defined using the following SQL statement:

```
CREATE TABLE Dept_Mgr (
    did INTEGER, dname CHAR(20), budget REAL, ssn CHAR(11), since DATE,
    PRIMARY KEY (did),
    FOREIGN KEY (ssn) REFERENCES Employees(ssn))
```

Translating Relationship Sets with Participation Constraints:

Total participation constraint insists that every department is required to have a manager, and key constraint says every department will have at most one manager. Participation constraint is implemented using NOT NULL constraint



```
CREATE TABLE Dept_Mgr(
    did INTEGER,
    dname CHAR(20),
    budget REAL,
    ssn CHAR(11) NOT NULL,
    since DATE,
    PRIMARY KEY (did),
    FOREIGN KEY (ssn) REFERENCES
    Employees(ssn),
    ON DELETE NO ACTION) );
```

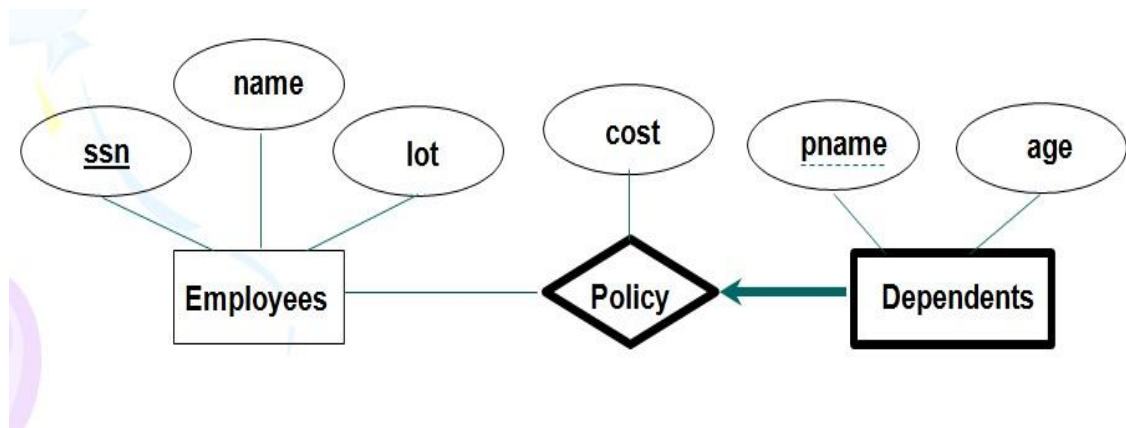
- It also captures the participation constraint that every department must have a manager:
- Because *ssn* cannot take on *null* values, each tuple of *Dept_Mgr* identifies a tuple in *Employees* (who is the manager).
- The NO ACTION specification, which is the default and need not be explicitly specified, ensures that an *Employees* tuple cannot be deleted while it is pointed to by a *Dept_Mgr* tuple.
- If we wish to delete such an *Employees* tuple, we must first change the *Dept_Mgr* tuple to have a new employee as manager.

Translating a weak entity set:

A weak entity set always participates in a one-to-many binary relationship and has a key constraint and total participation.

A *Dependents* entity can be identified uniquely only if we take the key of the *owning* *Employees* entity and the *pname* of the *Dependents* entity.

The *Dependents* entity must be deleted if the owning *Employees* entity is deleted.



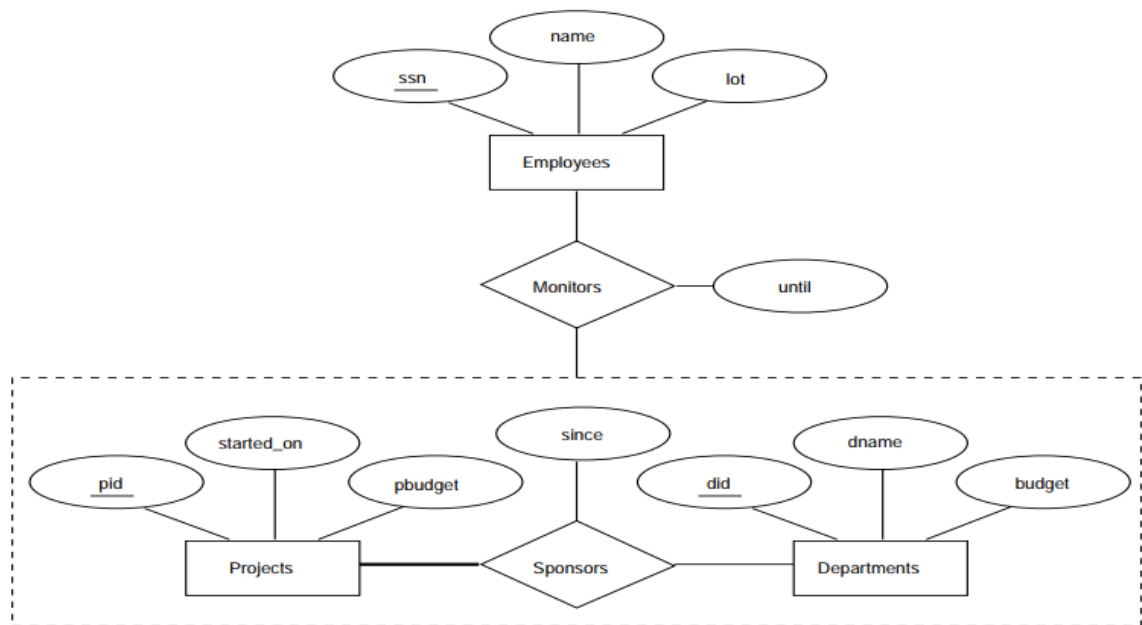
Weak entity set and identifying relationship set are translated into a single table.

When the owner entity is deleted, all owned weak entities must also be deleted.

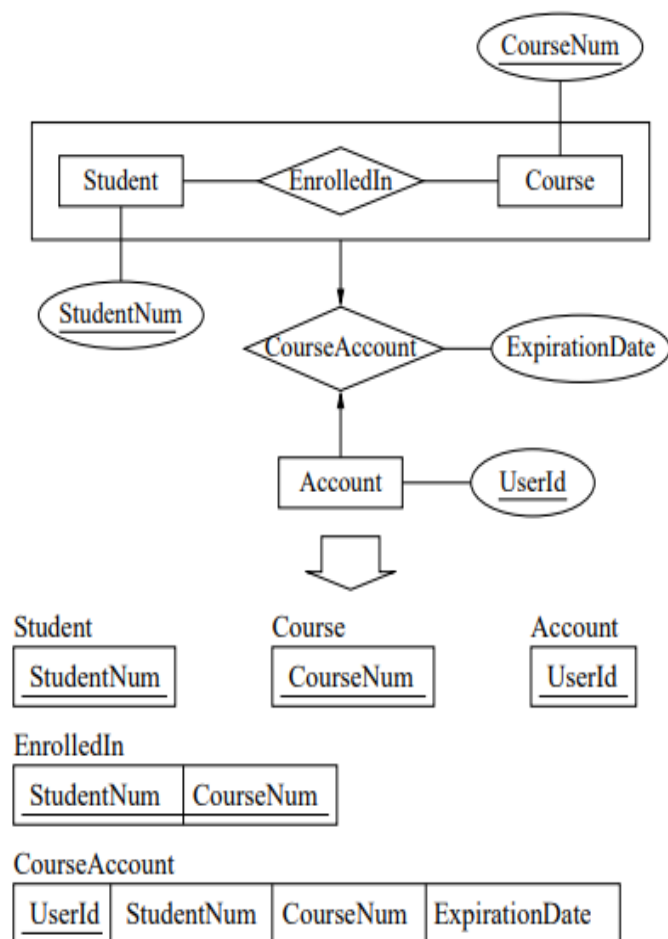
```
CREATE TABLE Dep_Policy (
  Pname      CHAR(20),
  age        INTEGER,
  cost       REAL,
  ssn        CHAR(11) NOT NULL,
  PRIMARY KEY (pname, ssn),
  FOREIGN KEY (ssn) REFERENCES Employees(ssn), ON DELETE CASCADE)
```

Translating ER Diagrams with Aggregation:

- For the ER diagram shown in Figure, the *Employees*, *Projects*, and *Departments* entity sets and the *Sponsors* relationship set are mapped as explained before.
- For the **Monitors relationship set**, we create a relation with the following attributes:
the key attributes of Employees (ssn), the key attributes of Sponsors (did, pid), and the descriptive attributes of Monitors (until).



Example:



DATABASE MANAGEMENT SYSTEMS
UNIT II – TOPIC 5
INTRODUCTION TO VIEWS

Views in SQL

- A view is a table whose rows are not explicitly stored in the database but are computed as needed from a view definition.
- A relation that is not of the conceptual model but is made visible to a user as a “virtual relation” is called a view.
- Views in SQL are considered as a virtual table. A view also contains rows and columns.
- To create the view, we can select the fields from one or more tables present in the database.
- A view can either have specific rows based on certain condition or all the rows of a table.
- Views can be used to present necessary information (or a summary), while hiding details in underlying relation(s) when data redundancy is to be kept minimum while maintaining security

1. Creating view

A view can be created using the **CREATE VIEW** statement. We can create a view from a single table or multiple tables.

Syntax:

```
CREATE VIEW view_name AS
    SELECT column1, column2.....
    FROM table_name
    WHERE condition;
```

Note: view_name: Name for the View,
table_name: Name of the table,
condition: Condition to select rows

Sample tables:

Student_Detail

STU_ID	NAME	ADDRESS
1	Stephan	Delhi
2	Kathrin	Noida
3	David	Ghaziabad
4	Alina	Gurugram

Student_Marks

STU_ID	NAME	MARKS	AGE
1	Stephan	97	19
2	Kathrin	86	21
3	David	74	18
4	Alina	90	20
5	John	96	18

Example : Creating View from a single table

In this example, we create a View named DetailsView from the table Student_Detail.

Query:

```
CREATE VIEW DetailsView AS
SELECT NAME, ADDRESS
FROM StudentDetails
WHERE S_ID < 4;
```

Just like table query, we can query the view to view the data.

```
SELECT * FROM DetailsView;
```

Output:

NAME	ADDRESS
Stephan	Delhi
Kathrin	Noida
David	Ghaziabad

Example : Creating View from multiple tables

View from multiple tables can be created by simply including multiple tables in the SELECT statement.

Syntax:

```
CREATE VIEW MarksView AS
SELECT Table1.colname1, Table1.colname2, Table2.colname3
FROM Table1,Table2
WHERE table1.colname4=table2.colname2;
```

In the given example, a view is created named MarksView from two tables Student_Detail and Student_Marks.

Query:

```
CREATE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS,
StudentMarks.MARKS
FROM StudentDetails, StudentMarks
WHERE StudentDetails.NAME = StudentMarks.NAME;
```

To display data of View MarksView:

```
SELECT * FROM MarksView;
```

Output:

NAME	ADDRESS	MARKS
Stephan	Delhi	97
Kathrin	Noida	86
David	Ghaziabad	74
Alina	Gurugram	90

2. Update SQL View:

The SQL view created can also be modified. We can do the following operations with the SQL VIEW.

But, **all views are not updatable**. A SQL view can be updated if the following conditions are satisfied.

1. The view is defined based on only one table.
2. The view should not have any field which is made of an **aggregate** function.
3. The view must not have GROUP BY, HAVING or DISTINCT clause in its definition.
4. The view should not be created using any nested query.
5. The selected output fields of the view must not use constants, string or value expressions.
6. If you want to update a view based on another view then that view should be updatable.

Updating a SQL View

We can use the **CREATE OR REPLACE VIEW** statement to **modify** the SQL view.

Syntax:

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1,column2,..
FROM table_name
WHERE condition;
```

Example: If we want to update the view marksview1 and remove the attribute "Age" from in the view then the query would be:

Query

```
mysql> create or replace view marksview1 as select student_detail.name,
student_marks.marks from student_detail, student_marks where
student_detail.std_id=student_marks.std_id;
Query OK, 0 rows affected (0.03 sec)
```

The above **CREATE OR REPLACE VIEW statement** would create a virtual table based on the result of the SELECT statement. Now, you can query the SQL VIEW as follows to see the output:

Output

```
mysql> select * from marksv1;
```

```
+-----+-----+
| name  | marks |
+-----+-----+
| stephan | 97 |
| kathrin | 86 |
| david  | 74 |
| alina  | 90 |
+-----+-----+
```

```
4 rows in set (0.03 sec)
```

Note::

- Views are dynamically generated, hence everytime user queries the view it fetches the information from the original table.
- Rows inserted into an updatable view will insert the corresponding row into the original table.
- Attempts to insert rows through a view that doesnot contain Primary Key will be rejected as PK cannot take Null values.
- When views are created based on selection, the insertions may reflect in base table but may not be present in the view.

Deleting View

A view can be deleted using the Drop View statement.

Syntax

```
DROP VIEW view_name;
```

View_name: Name of the View which we want to delete.

Example:

If we want to delete the View **MarksView**, we can do this as:

```
EX: DROP VIEW MarksView;
```

NOTE: A view will also be dropped if a table is dropped using the CASCADE option.

Example : Drop table Student CASCADE;

However, a drop table with restrict can be used to prevent accidental deletions.

```
Drop table Student RESTRICT
```

will not drop student table if there is a view or integrity constraint refers to Student.

Executed Queries on Views:

Creating and inserting data into student_details table:

```
mysql> create table student_detail(std_id int,name varchar(20),address varchar(20));
```

Query OK, 0 rows affected (0.33 sec)

```
mysql> insert into student_detail(std_id,name,address) values
```

```
(1,'stephan','delhi'),(2,'kathrin','noida'),(3,'david','ghaziabad'),(4,'alina','gurugram');
```

Query OK, 4 rows affected (0.11 sec)

Records: 4 Duplicates: 0 Warnings: 0

Creating and inserting data into student_marks table:

```
mysql> create table student_marks(std_id int,name varchar(20),marks int,age int);
```

Query OK, 0 rows affected (0.11 sec)

```
mysql> insert into student_marks(std_id,name,marks,age) values
```

```
(1,'stephan',97,19),(2,'kathrin',86,21),(3,'david',74,18),(4,'alina',90,19),(5,'john',96,18);
```

Query OK, 5 rows affected (0.02 sec)

Records: 5 Duplicates: 0 Warnings: 0

Creating view detailsview on student_detial:

```
mysql> create view detailsview as select name,address from student_detail where std_id<4;
```

Query OK, 0 rows affected (0.17 sec)

```
mysql> select * from detailsview;
```

```
+-----+-----+
```

```
| name | address |
```

```
+-----+-----+
```

```
| stephan | delhi |
```

```
| kathrin | noida |
```

```
| david | ghaziabad |
```

```
+-----+-----+
```

3 rows in set (0.08 sec)

Creating view marksviiew on student_detial and s=student_marks:

```
mysql> create view marksviiew as select student_detail.name, student_detail.address,
student_marks.marks from student_detail, student_marks where
student_detail.name=student_marks.name;
```

Query OK, 0 rows affected (0.09 sec)

```
mysql> select * from marksviiew;
```

```
+-----+-----+-----+
```

```
| name | address | marks |
```

```
+-----+-----+-----+
```

```
| stephan | delhi | 97 |
```

```
| kathrin | noida | 86 |
```

```
| david | ghaziabad | 74 |
```

```
| alina | gurugram | 90 |
```

```
+-----+-----+-----+
```

4 rows in set (0.00 sec)

Deleting View marksvew:

```
mysql> drop view marksvew;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> create view marksvew1 as select student_detail.name,
student_marks.marks,student_marks.age from student_detail, student_marks where
student_detail.std_id=student_marks.std_id;
```

Query OK, 0 rows affected (0.20 sec)

```
mysql> select * from marksvew1;
```

```
+-----+-----+-----+
```

```
| name | marks | age |
```

```
+-----+-----+-----+
```

```
| xyz | 98 | 18 |
```

```
| pqr | 95 | 19 |
```

```
| abc | 68 | 17 |
```

```
| uvw | 78 | 18 |
```

```
+-----+-----+-----+
```

4 rows in set (0.09 sec)

DATABASE MANAGEMENT SYSTEMS
UNIT II – TOPIC 6
ER-RELATIONAL – ADDITIONAL EXAMPLES

There are 4 types of relationships:

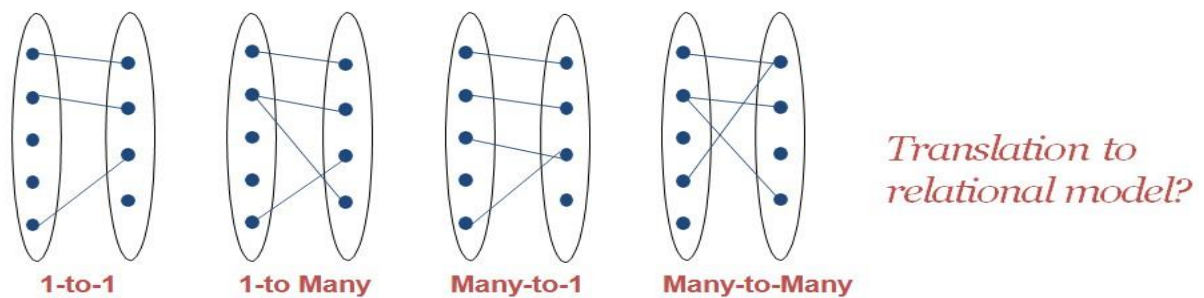
1:1 [One-to-One]

1:M [One-to-Many]

M:1 [Many-to-One].

M: M [Many-to-Many]

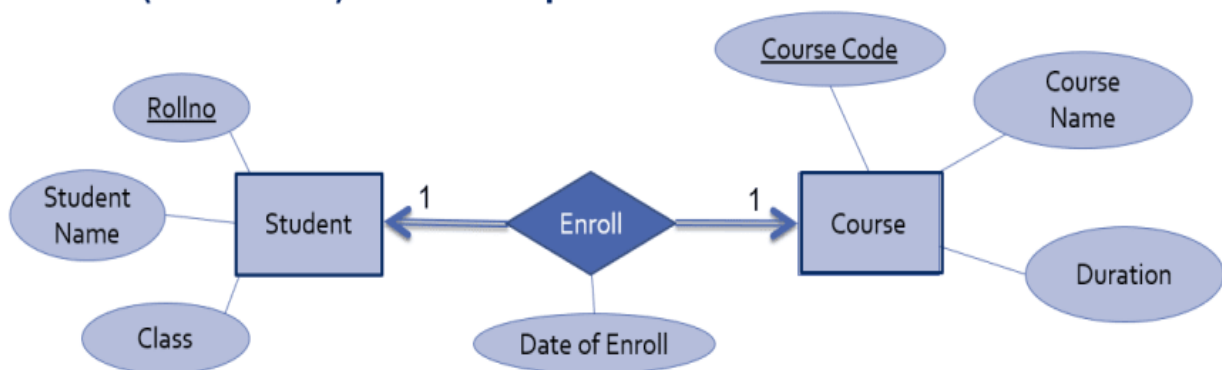
Let us see the mapping of these relationship sets into tables



1:1 (One to One) Relationship:

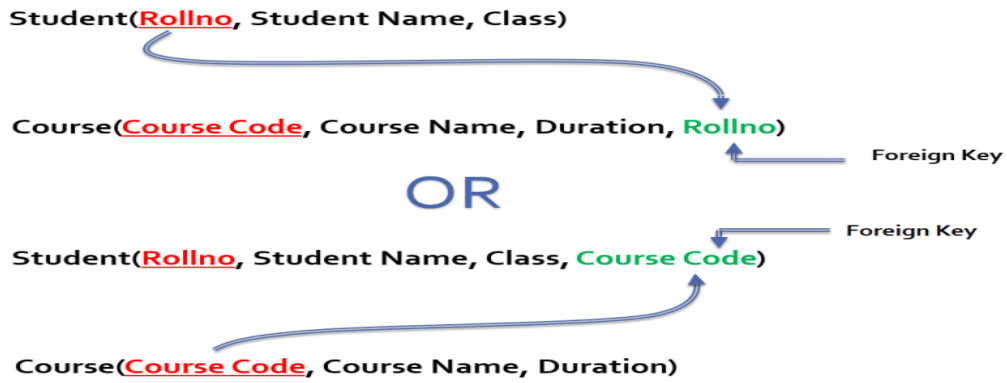
Consider 1:1 relationship set enroll exist between entity sets student and course, which means one student can enroll in only one courses

1:1 (One to one) Relationship



To convert this Relationship set into relational schema,

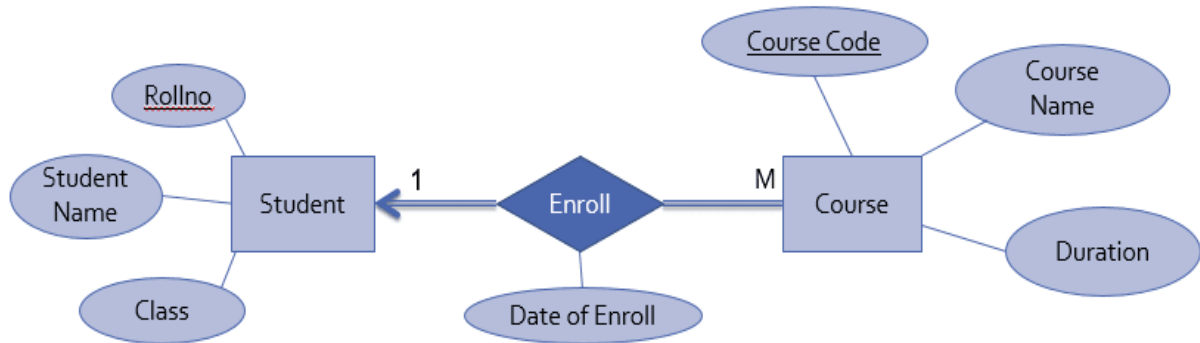
1. Separate relation is created for each participating entity sets.
2. Primary Key of Relation Student can be act as foreign key for relation Course
OR
Primary Key of Relation Course act as foreign key for relation Student.



1:M (One to Many) Relationship:

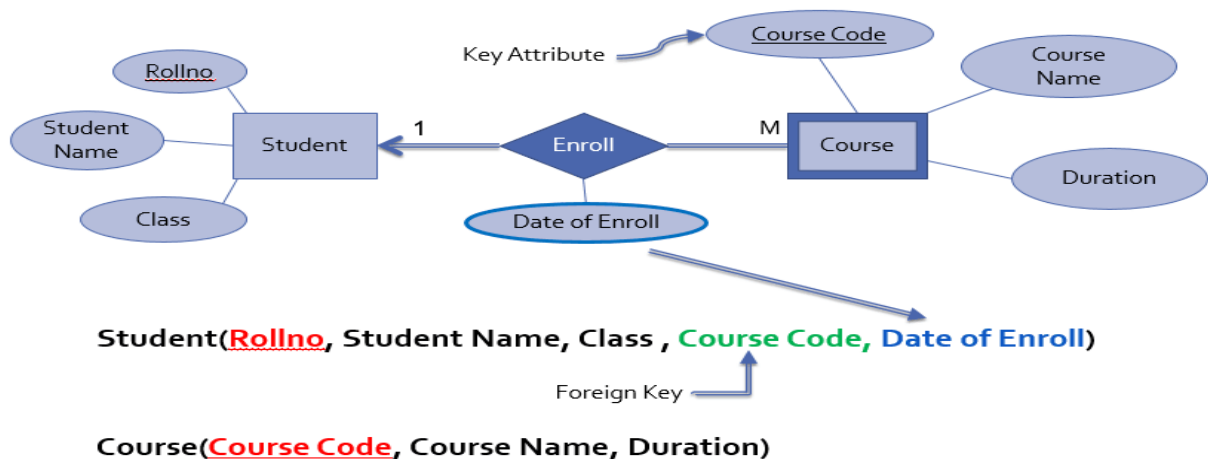
Consider 1:M relationship set enrolled exist between entity sets Student and Course which means that one student can enroll in multiple courses.

1:M (one to many) Relationship



In this case to convert this relationship into relational schema,

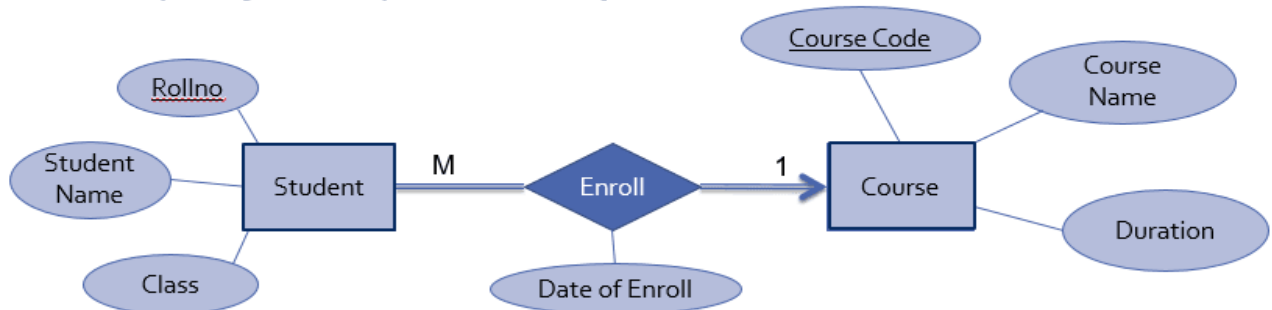
1. Separate relation is created for each participating entity sets (student and course)
2. Key attribute of Many's side entity set (course) is mapped as foreign key in one's side relation(Student)
3. All attributes of relationship set are mapped as attributes for relation of one's side entity set (student)



M:1 (Many to One) Relationship:

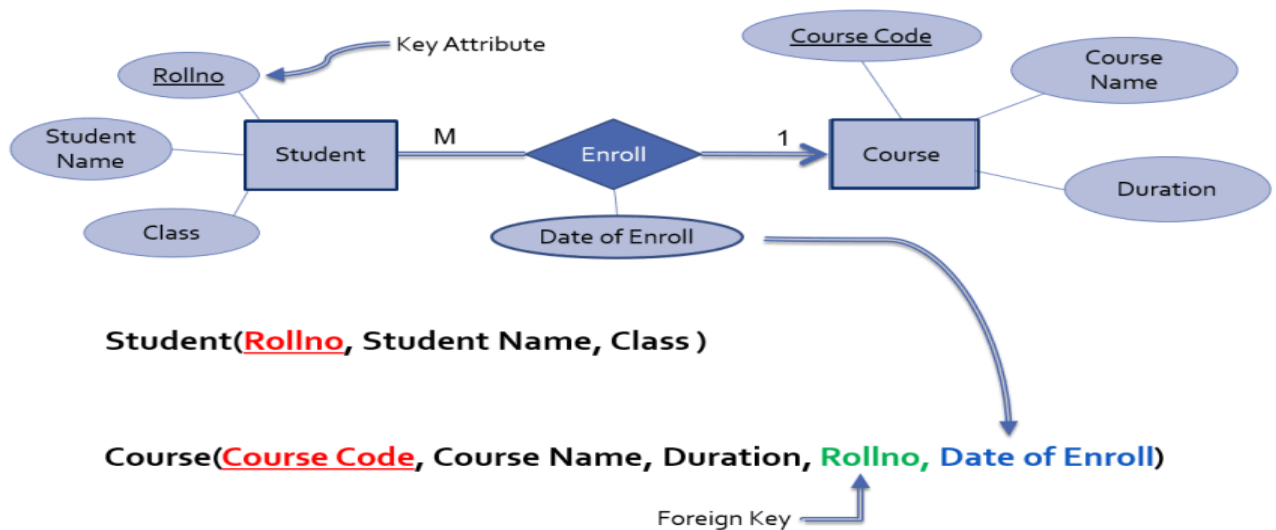
Consider relationship set enroll exist between entity sets student and course . but here student is many side entity set while course is one side entity set. Which means many students can enroll in one course.

M:1 (many to one) Relationship



To convert this relationship set into relational schema,

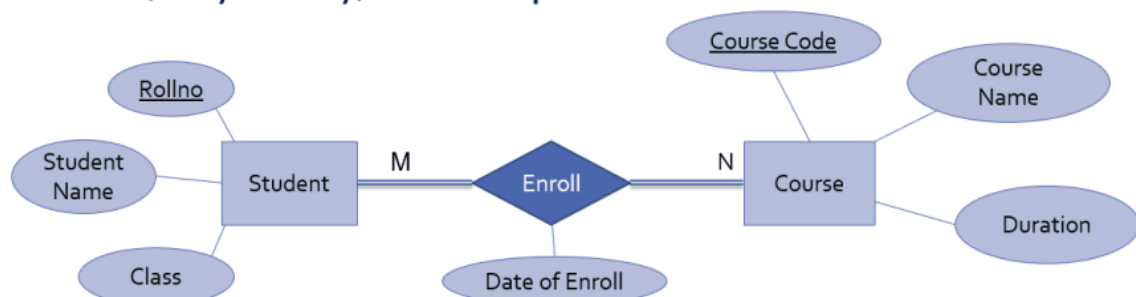
1. Separate relation is created for each participating entity sets.
2. Key attribute of Many's side entity set student is mapped as foreign key in one's side relation
3. All attributes of relationship set are mapped as attributes for one's side relation course.



M:N (Many to Many) Relationship:

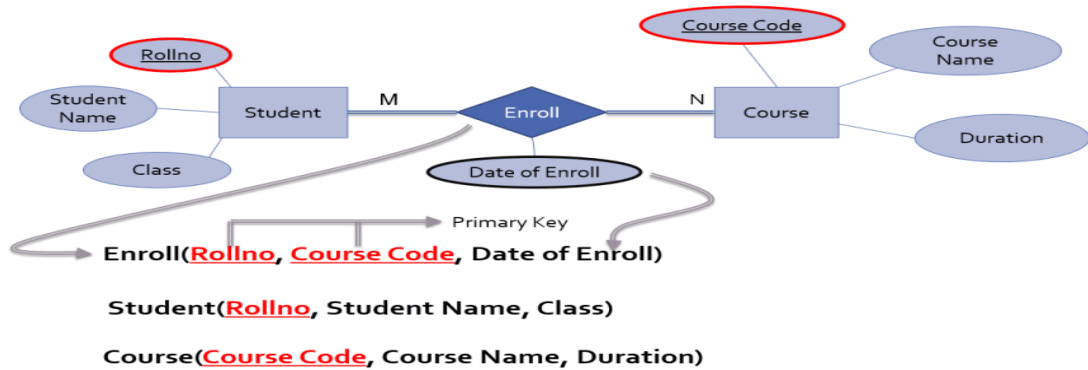
Consider same relationship set enrolled exist between entity sets student and course, which means multiple student can enroll in multiple courses.

M:N (many to many) Relationship

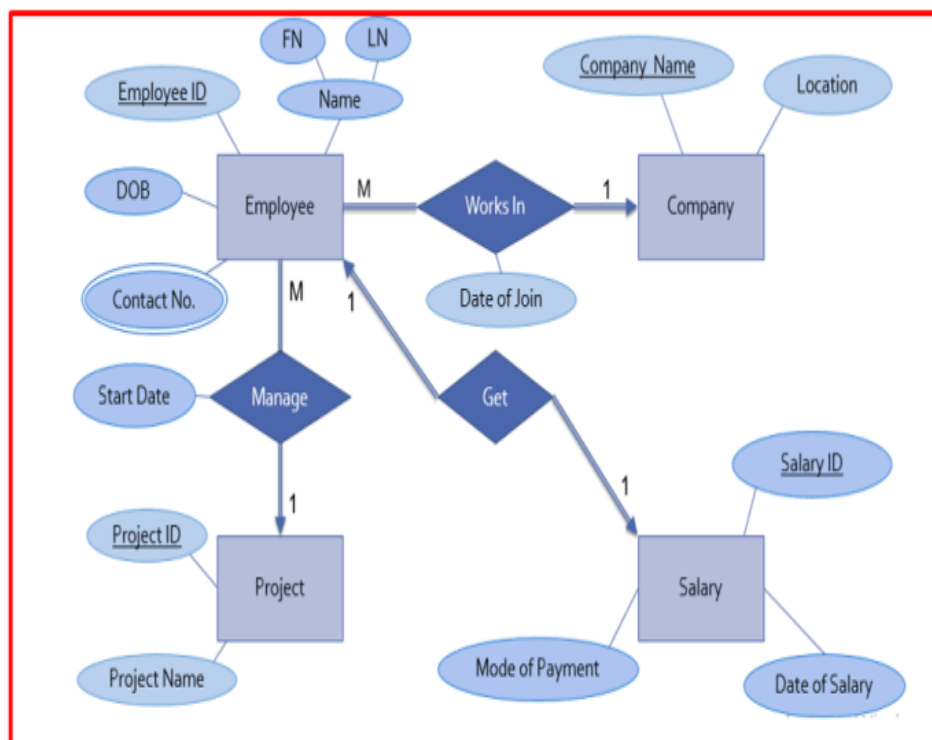


To convert this Relationship set into relational schema,

1. Relationship set is mapped as separate relation
2. Key attributes of participating entity sets are mapped as primary key for that relation
3. Attribute of relationship set becomes simple attributes for that relation
4. And separate relation is created for other participating entities



EXAMPLE 1:



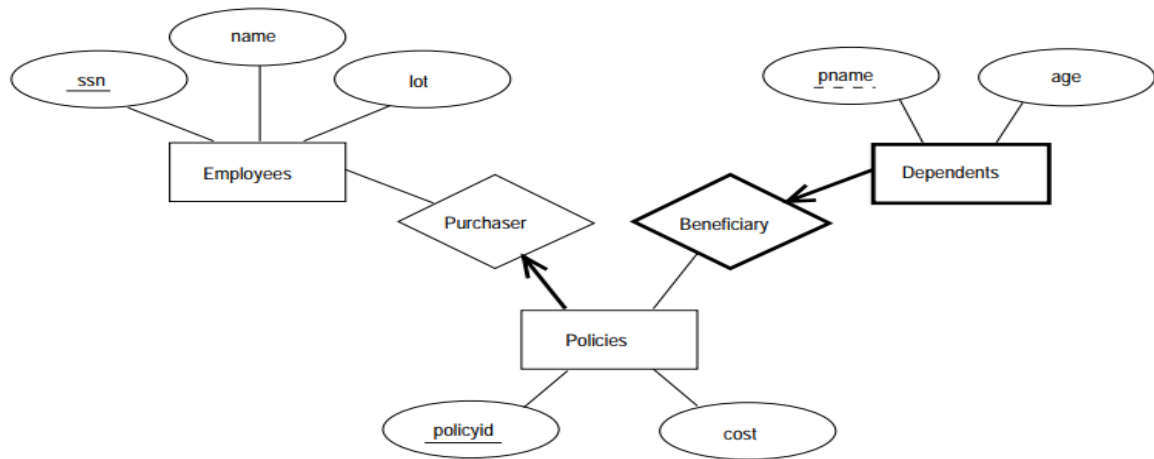
ER Diagram

Employee(Employee ID , DOB , FN , LN , Salary ID)
EmployeeContact(Employee ID , Contact No)
Company(Company Name , Location , Employee ID , Date of Join)
Project(Project ID , Project Name , Employee ID , Start Date)
Salary(Salary ID , Mode of Payment , Date of Salary , Employee ID)

Relational Model

EXAMPLE 2:

Consider the ER diagram shown in Figure below. We can translate this ER diagram into the relational model as follows, taking advantage of the key constraints to combine Purchaser information with Policies and Beneficiary information with Dependents:



```
CREATE TABLE Policies ( policyid INTEGER,  
                        cost REAL,  
                        ssn CHAR(11) NOT NULL,  
                        PRIMARY KEY (policyid),  
                        FOREIGN KEY (ssn) REFERENCES Employees ON DELETE CASCADE ) ;
```

```
CREATE TABLE Dependents ( pname CHAR(20),  
                           age INTEGER,  
                           policyid INTEGER,  
                           PRIMARY KEY (pname, policyid),  
                           FOREIGN KEY (policyid) REFERENCES Policies ON DELETE CASCADE ) ;
```

Note: the deletion of an employee leads to the deletion of all policies owned by the employee and all dependents who are beneficiaries of those policies